

CXL RAS Error Injection

Guest-Side CCI Injection, CCI Mailbox Theory and Full Results

1. Background

RAS (Reliability, Availability, Serviceability) is the umbrella term for how a system detects hardware problems, keeps running despite them, and logs enough detail for diagnosis. This document covers the testing of RAS behaviour of a CXL memory device under different kinds of injected faults.

QEMU inside WSL2/Linux was used to emulate a CXL device, as real CXL hardware is expensive and rare.

2. Environment

Item	Detail
Host	Windows 11, WSL2 (Ubuntu 24.04) / Linux
Kernel (host/WSL2)	Custom build from microsoft/WSL2-Linux-Kernel, branch linux-msft-wsl-6.6.y, with KVM and ACPI_APEI_EINJ enabled
QEMU	jic23 fork (gitlab.com/jic23/qemu), branch cxl-2025-03-20, with 3 patches applied for CXL RAS
Guest kernel	torvalds/linux v6.18, custom config with CXL_BUS, CXL_MEM, CXL_REGION, ACPI_APEI_GHES, PCIEAER, X86_MCE etc

3. What Was Already Done

Error Type	Levels	IRQ	Seen In	Status
cxl_general_media	info/warn/fail/fatal	38-41	trace_pipe	Done
cxl_dram	info/warn/fail/fatal	38-41	trace_pipe	Done
cxl_memory_module	4 levels, escalating health	38-41	trace_pipe	Done
cxl_poison	—	none	trace buffer	Done
Correctable AER	mem-data-ecc	25	dmesg	Done
Uncorrectable Fatal AER	mem-data-ecc	25	dmesg	Done

4. QMP Injection

QMP (QEMU Monitor Protocol) is a JSON-based control interface for QEMU itself, running on the host (WSL2/Linux). It allows an external program to send commands directly to QEMU's own internal model of the hardware not to the guest OS and not to the emulated device's firmware.

4.1 What Happens When a QMP Injection Command Is Sent

Example command for correctable AER:

```
{ "execute": "qmp-cxl-inject-correctable-errors",  
  "arguments": { "path": "/machine/peripheral/cxl1",  
                 "errors": [ { "type": "mem-data-ecc" } ] } }
```

Step by step:

1. Sending the Command

Send a JSON-formatted command over a network connection (127.0.0.1:4444). QEMU was listening on this port because it was configured to do so when it started.

2. QEMU Processes the Command

QEMU receives the command and passes it to its internal QMP command handler. The handler identifies the requested operation and calls the corresponding C function.

3. Updating the Virtual Device State

The C function locates the virtual CXL device within QEMU and directly updates its internal memory structures. Instead of simulating a physical fault, it modifies the device's status information to indicate that an error has occurred.

4. Simulating an Error Condition

By changing the device state directly, QEMU makes the virtual hardware appear as though a real error has happened. The end result is the same as a hardware fault, but without modeling the physical process that would normally cause it.

5. Guest OS Handles the Error

Once the error state is set, the virtual hardware generates the appropriate notification or interrupt. The guest Linux operating system receives this event, detects the error through its standard drivers, and responds exactly as it would on a physical system.

4.2 Where QMP Sits in the Overall Error Pathway

Every real CXL error moves through five stages:

- (1) fault in the device's memory cells,
- (2) device firmware detects and records it internally
- (3) that internal state becomes visible to the host
- (4) the guest kernel processes it
- (5) a user-visible outcome appears.

QMP skips straight to stage two it fabricates the internal state by hand, with no real fault behind it. Stages three through five then run for real, as the guest kernel cannot distinguish between firmware-set bits and QEMU-written bits.

5. CCI and the CXL Mailbox

5.1 What Is CCI

CCI is the command/response protocol defined by the CXL specification that lets software talk to a CXL device's internal controller.

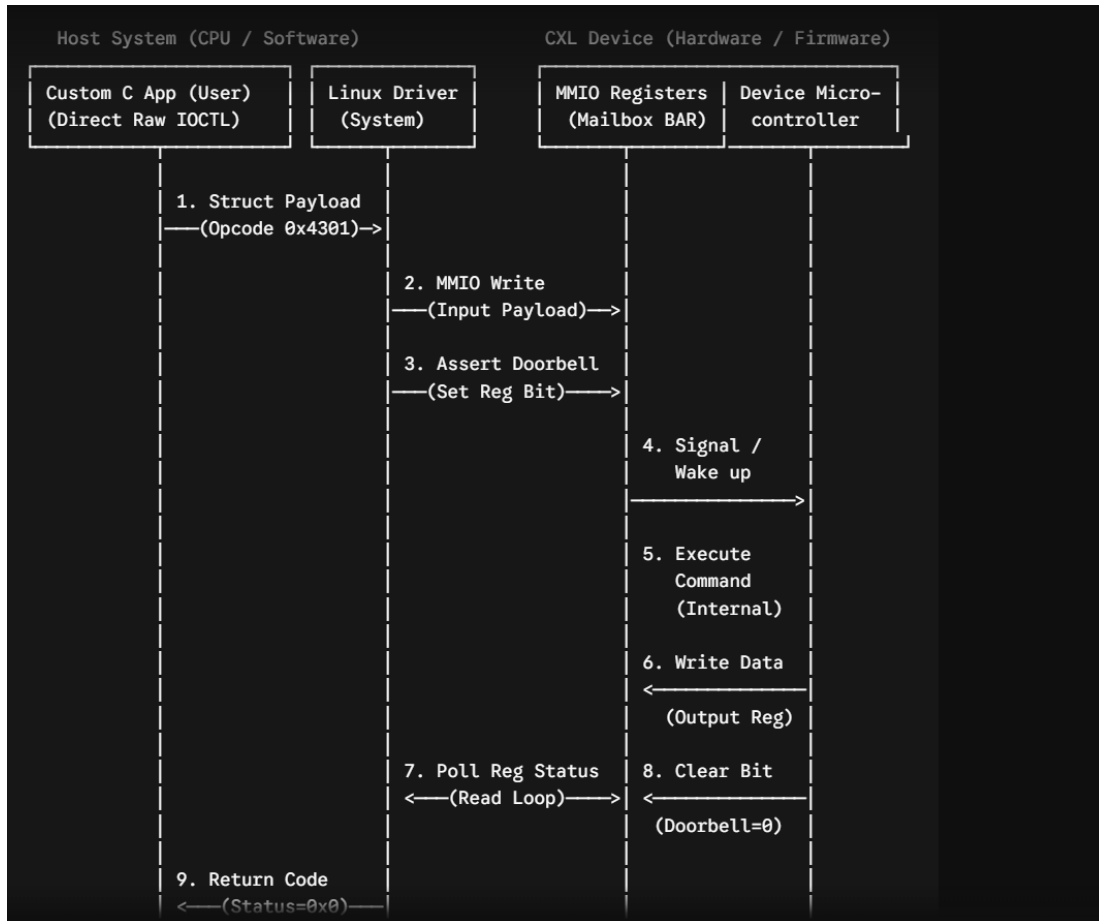
Every CCI interaction follows the same pattern: software writes a command (opcode plus input data) into the device's mailbox registers, tells the device to execute it, waits for completion, then reads the result back out of the mailbox.

5.2 What Is the Mailbox

The mailbox is the actual hardware mechanism CCI runs over a small region of registers/memory exposed by the CXL device, mapped into the host's address space via the device's PCIe BAR. It has these components:

- Input payload area — where the host writes the command's input data.
- Output payload area — where the device writes command results.
- Command register — where the host writes the opcode, then signals 'go'.
- Status/return code register — where the device reports success or an error code once done.

5.3 The Eight-Step Synchronous Protocol



- Read the Mailbox Control Register to verify the **doorbell** is clear, indicating that the mailbox is not busy.
- Write the Command Register with the desired opcode (for example, 0x4301 for Inject Poison) and the input payload length.
- Write the Command Payload Registers if the command requires input data. For Inject Poison, this consists of the 8-byte DPA address.
- Set the doorbell bit in the Mailbox Control Register to submit the command. This is the actual send operation; all previous steps were preparation.
- Wait for the doorbell bit to clear, either by **polling** the register or by receiving an interrupt if one is configured. During this time, the device processes the command.
- Read the Mailbox Status Register to obtain the return code. A value of 0 indicates success, while non-zero values indicate specific errors such as Unsupported Command or Invalid Input.
- If the command completed successfully, read the Command Register again to determine the output payload length.
- If output data is available, read it from the Command Payload Registers.

5.4 How CCI Differs from QMP

Aspect	QMP Injection	CCI / Mailbox Injection
Where command originates	Host side (WSL2/Linux), sent to QEMU's control socket	Guest side — inside the VM, by a kernel driver or userspace program
What receives the command	QEMU's own C code (the emulator process)	The emulated CXL device's mailbox registers, processed by QEMU's emulated mailbox dispatcher
What gets modified first	QEMU's internal Object state directly	The mailbox command/status/payload registers
Realism	Low simulates the outcome without running the pipeline	Comparatively better

6. CXL Poison

Poison, in CXL terms, is a marker attached to a specific address (a DPA Device Physical Address) indicating that the data at that location is known to be bad. It does not fix or move the data; it is a flag that travels with that memory location so that anything reading it later is warned.

Poison is the mechanism that turns a silent data-corruption problem into a detectable one. Without it, a bad memory cell might quietly return wrong bits with no indication until the corrupted value caused a downstream failure.

With poison, the moment something tries to read that address, the hardware immediately signals 'this is poisoned' rather than handing back silently-wrong data.

7. Injections

7.1 CXL_Poison_Debugfs.sh

CXL_Poison_Debugfs.sh uses the kernel's debugfs interface to inject CXL poison. It enables kernel tracing, writes the target DPA to the inject_poison debugfs file, triggers a Get Poison List command, and reads the trace buffer to confirm the kernel recorded the event end to end.

```
#!/bin/bash

set -e

TARGET_DEV="mem0"
TARGET_DPA="0x10000000"

CXL_DEBUG_INJECT="/sys/kernel/debug/cxl/$TARGET_DEV/inject_poison"
CXL_DEBUG_CLEAR="/sys/kernel/debug/cxl/$TARGET_DEV/clear_poison"
CXL_SYS_TRIGGER="/sys/bus/cxl/devices/$TARGET_DEV/trigger_poison_list"
TRACE_BUFFER="/sys/kernel/debug/tracing/trace"
TRACE_ENABLE="/sys/kernel/debug/tracing/events/cxl/cxl_poison/enable"

# Check for root privileges
if [ "$EUID" -ne 0 ]; then
    echo "[-] Error: Please run this script as root."
```

```

    exit 1
fi

echo "[+] Step 1: Configuring OS kernel tracing infrastructure"
echo 1 > "$TRACE_ENABLE"
echo 0 > "$TRACE_BUFFER"

echo "[+] Step 2: Injecting CXL Media Error via CCI Mailbox (Opcode 0x4301)"
if [ -f "$CXL_DEBUG_INJECT" ]; then
    echo "$TARGET_DPA" > "$CXL_DEBUG_INJECT"
    echo " -> Successfully sent Inject Poison payload for DPA $TARGET_DPA"
else
    echo "[-] Error: CCI injection interface missing. Check kernel configuration."
    exit 1
fi

echo "[+] Step 3: Triggering Hardware 'Get Poison List' over CCI (Opcode 0x4100)"
if [ -f "$CXL_SYS_TRIGGER" ]; then
    echo 1 > "$CXL_SYS_TRIGGER"
    # Allow a split second for hardware mailbox handling
    sleep 0.5
else
    echo "[-] Warning: Standard trigger missing, attempting fallback region walk"
    cat /sys/bus/cxl/devices/region*/poison > /dev/null 2>&1 || true
fi

echo "[+] Step 4: Extracting hardware-reported telemetry..."
if grep -q "dpa=$TARGET_DPA" "$TRACE_BUFFER"; then
    echo ""
    echo " SUCCESS: Hardware CCI confirmed error state persistence!"
    echo ""
    grep "dpa=$TARGET_DPA" "$TRACE_BUFFER"
else
    echo "[-] Error: Injected address not found in hardware poison logs."
    exit 1
fi

# Optional Step 5: Cleanup/Clear Poison
# echo "[+] Step 5: Cleaning up hardware state via CCI Clear Poison (Opcode 0x4102)..."
# echo "$TARGET_DPA" > "$CXL_DEBUG_CLEAR"

echo "[+] Done."

```

Output:

```

ubuntu@ubuntu:~$ chmod +x CXL_Poison_Debugfs
ubuntu@ubuntu:~$ sudo ./CXL_Poison_Debugfs
[+] Step 1: Configuring OS kernel tracing infrastructure...
[+] Step 2: Injecting CXL Media Error via CCI Mailbox (Opcode 0x4301)...
-> Successfully sent Inject Poison payload for DPA 0x10000000
[+] Step 3: Triggering Hardware 'Get Poison List' over CCI (Opcode 0x4100)...
[+] Step 4: Extracting hardware-reported telemetry...
=====
SUCCESS: Hardware CCI confirmed error state persistence!
=====
CXL_Poison_Debu-667 [003] ..... 116.739446: cxl_poison: memdev=mem0 host=0000:0d:00.0 serial=1 trace_type=Inject region= region_uuid=00000000-0000-00
00-0000-000000000000 hpa=0xffffffffffffffff hpa_alias=0xffffffffffffffff dpa=0x10000000 dpa_length=0x40 source=Unknown flags= overflow_time=0
CXL_Poison_Debu-667 [003] ..... 116.740795: cxl_poison: memdev=mem0 host=0000:0d:00.0 serial=1 trace_type=List region= region_uuid=00000000-0000-0000
-0000-000000000000 hpa=0xffffffffffffffff hpa_alias=0xffffffffffffffff dpa=0x10000000 dpa_length=0x40 source=Injected flags= overflow_time=0
[+] Done.
ubuntu@ubuntu:~$

```

7.2 CXL_Raw_Poison_CCI.sh (Direct Raw CCI Mailbox Injection)

CXL_Raw_Poison_CCI.sh talks directly to the CXL device mailbox via ioctl, bypassing the kernel driver entirely. Unlike the debugfs approach in 7.1 where the kernel issues the mailbox command on our behalf here the program constructs the CCI command itself, setting opcode 0x4301 to inject poison at a target DPA and 0x4302 to clear it.

```
#!/bin/bash

set -e

DEVICE="/dev/cxl/mem1"
RAW_ALLOW="/sys/kernel/debug/cxl/mbox/raw_allow_all"
WORKDIR="$(mktemp -d)"
SRC="$WORKDIR/cxl_raw_poison.c"
BIN="$WORKDIR/cxl_raw_poison"

cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT

if [ "$EUID" -ne 0 ]; then
    echo "[-] Error: Please run this script as root."
    exit 1
fi

if ! command -v gcc >/dev/null 2>&1; then
    echo "[-] Error: gcc not found. Install a compiler (e.g. apt install gcc)."
    exit 1
fi

if [ ! -e "$DEVICE" ]; then
    echo "[-] Error: $DEVICE not found. Is the CXL device attached/named mem1?"
    exit 1
fi

echo "[+] Step 1: Enabling raw CCI mailbox commands"
if [ -f "$RAW_ALLOW" ]; then
    echo Y > "$RAW_ALLOW"
else
    echo "[-] Warning: $RAW_ALLOW not present; continuing, ioctl may be rejected."
fi

echo "[+] Step 2: Writing and compiling raw CCI mailbox injector"
cat > "$SRC" << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdint.h>
#include <sys/ioctl.h>

#define CXL_MEM_SEND_COMMAND_IOWR(0xCE, 2, struct cxl_send_command)
#define CXL_MEM_COMMAND_ID_RAW 2

#define CXL_MBOX_OP_INJECT_POISON 0x4301
#define CXL_MBOX_OP_CLEAR_POISON 0x4302

struct cxl_send_command {
    uint32_t id; uint32_t flags;
    union { struct { uint16_t opcode; uint16_t rsvd; } raw; uint32_t rsvd; };
    uint32_t retval;
```

```

struct { uint32_t size; uint32_t rsvd; uint64_t payload; } in;
struct { uint32_t size; uint32_t rsvd; uint64_t payload; } out;
};

int send_raw(int fd, uint16_t opcode, void *in_buf, uint32_t in_size) {
    struct cxl_send_command cmd = {0};
    cmd.id = CXL_MEM_COMMAND_ID_RAW; cmd.raw.opcode = opcode;
    if (in_buf) { cmd.in.size = in_size; cmd.in.payload = (uint64_t)(uintptr_t)in_buf; }
    int ret = ioctl(fd, CXL_MEM_SEND_COMMAND, &cmd);
    if (ret < 0) { perror("ioctl"); return -1; }
    printf("    retval: 0x%x %s\n", cmd.retval,
           cmd.retval == 0 ? "(SUCCESS)" : "(ERROR)");
    return cmd.retval;
}

int main() {
    int fd = open("/dev/cxl/mem1", O_RDWR);
    if (fd < 0) { perror("/dev/cxl/mem1"); return 1; }
    printf("=== CXL Raw CCI Poison via opcode 0x4301 ===\n\n");

    uint64_t *dpa = calloc(1, sizeof(uint64_t));
    if (!dpa) { perror("calloc"); close(fd); return 1; }
    *dpa = 0x50000;

    printf("[*] INJECT_POISON (0x4301) at DPA 0x%lx:\n", *dpa);
    int ret = send_raw(fd, CXL_MBOX_OP_INJECT_POISON, dpa, sizeof(*dpa));
    if (ret == 0) {
        printf("[+] Poison injected via raw CCI mailbox!\n\n");
        uint8_t clear_pl[72] = {0};
        memcpy(clear_pl, dpa, 8);
        printf("[*] CLEAR_POISON (0x4302) at DPA 0x%lx:\n", *dpa);
        send_raw(fd, CXL_MBOX_OP_CLEAR_POISON, clear_pl, sizeof(clear_pl));
    }
    free(dpa);
    close(fd);
    return 0;
}
EOF

gcc -O2 -o "$BIN" "$SRC"

echo "[+] Step 3: Running raw CCI mailbox injector against $DEVICE"
"$BIN"

echo "[+] Done."

```

A userspace program inside the guest talking directly to the CXL device's firmware mailbox, using the documented `ioctl` with correct opcodes taken from the kernel source, successfully injecting and clearing poison with no debugfs wrapper, no AER layer and no QMP

Output:

```
Expanded Security Maintenance for Applications is not enabled.

8 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

ubuntu@ubuntu:~$ sudo ./CXL_Raw_Poison_CCI
[+] Step 1: Enabling raw CCI mailbox commands
[+] Step 2: Writing and compiling raw CCI mailbox injector
[+] Step 3: Running raw CCI mailbox injector against /dev/cxl/mem1
=== CXL Raw CCI Poison via opcode 0x4301 ===

[*] INJECT_POISON (0x4301) at DPA 0x50000:
retval: 0x0 (SUCCESS)
[+] Poison injected via raw CCI mailbox!

[*] CLEAR_POISON (0x4302) at DPA 0x50000:
retval: 0x0 (SUCCESS)
[+] Done.
ubuntu@ubuntu:~$ dmesg | grep -i cxl
[ 0.000000] Command line: root=/dev/vda1 console=ttyS0 console=tty1 cxl_acpi.cxl_res_enable=1
[ 0.026370] Kernel command line: root=/dev/vda1 console=ttyS0 console=tty1 cxl_acpi.cxl_res_enable=1
[ 0.781360] acpi ACPI0016:00: _OSC: OS supports [CXL11PortRegAccess CXL20PortDevRegAccess CXLProtocolErrorReporting]
[ 0.784359] acpi ACPI0016:00: _OSC: OS now controls [CXLMemErrorReporting]
[ 0.881470] cxl root0: Extended linear cache calculation failed rc:-2
[ 0.884378] pci0000:0c: host supports CXL
[ 29.187241] cxl_pci 0000:0d:00:0: raw command path used
[ 29.187264] WARNING: CPU: 3 PID: 661 at drivers/cxl/core/mbox.c:416 cxl_send_cmd+0x4a1/0x5a0
[ 29.187318] CPU: 3 UID: 0 PID: 661 Comm: cxl_raw_poison Not tainted 6.18.0 #4 PREEMPT(voluntary)
[ 29.187321] RIP: 0010:cxl_send_cmd+0x4a1/0x5a0
[ 29.187377] cxl_memdev_ioctl+0x52/0xa0
ubuntu@ubuntu:~$
```

7.3 AER_Inject_Correctable.sh

aer_inject_write.c injects a correctable PCIe AER error directly from inside the guest using the kernel's aer-inject module. It opens /dev/aer_inject and writes a structured error descriptor targeting 0000:0e:00.0 (the CXL device on mem1) with error type PCI_ERR_COR_BAD_DLLP (0x0080). The injection runs entirely from the guest side with no QEMU host involvement. The kernel receives it, logs it and handles it silently since correctable errors require no recovery action.

```
#!/bin/bash

set -e

WORKDIR="$(mktemp -d)"
SRC="$WORKDIR/aer_inject_write.c"
BIN="$WORKDIR/aer_inject_write"

cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT

if [ "$EUID" -ne 0 ]; then
    echo "[-] Error: Please run this script as root."
    exit 1
fi

if ! command -v gcc >/dev/null 2>&1; then
    echo "[-] Error: gcc not found. Install a compiler (e.g. apt install gcc)."
    exit 1
fi

echo "[+] Step 1: Ensuring aer-inject kernel module is loaded..."
if ! lsmod | grep -q '^aer_inject'; then
    modprobe aer-inject || {
        echo "[-] Error: could not load aer-inject module."
        exit 1
    }
fi

if [ ! -e /dev/aer_inject ]; then
```

```
    echo "[+] Error: /dev/aer_inject not found even after modprobe."
    exit 1
fi
```

```
echo "[+] Step 2: Writing and compiling correctable AER injector."
```

```
cat > "$SRC" << 'EOF'
```

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdint.h>
```

```
struct aer_error_inj {
    uint8_t bus; uint8_t dev; uint8_t fn;
    uint32_t uncor_status; uint32_t cor_status;
    uint32_t header_log0; uint32_t header_log1;
    uint32_t header_log2; uint32_t header_log3;
    uint32_t domain;
};
```

```
#define PCI_ERR_COR_BAD_DLLP 0x0080
```

```
int main() {
    int fd = open("/dev/aer_inject", O_WRONLY);
    if (fd < 0) { perror("open /dev/aer_inject"); return 1; }
    struct aer_error_inj einj;
    memset(&einj, 0, sizeof(einj));
    einj.bus = 0x0e; einj.dev = 0x00; einj.fn = 0x00;
    einj.cor_status = PCI_ERR_COR_BAD_DLLP; einj.domain = 0;
    printf("[*] Injecting correctable AER on 0000:0e:00.0\n");
    ssize_t ret = write(fd, &einj, sizeof(einj));
    if (ret < 0) perror("write");
    else printf("[*] Done. Bytes written: %zd\n", ret);
    close(fd);
    return ret < 0 ? 1 : 0;
}
```

```
EOF
```

```
gcc -O2 -o "$BIN" "$SRC"
```

```
echo "[+] Step 3: Running correctable AER injector"
"$BIN"
```

```
echo "[+] Step 4: Check kernel log for the correctable AER report:"
echo "    dmesg | grep -i aer"
```

```
echo "[+] Done."
```

Output:

```
ubuntu@ubuntu:~$ chmod +x AER_Inject_Correctable
ubuntu@ubuntu:~$ sudo ./AER_Inject_Correctable
[+] Step 1: Ensuring aer-inject kernel module is loaded...
[+] Step 2: Writing and compiling correctable AER injector...
[+] Step 3: Running correctable AER injector...
[*] Injecting correctable AER on 0000:0e:00.0...
[*] Done. Bytes written: 32
[+] Step 4: Check kernel log for the correctable AER report:
dmesg | grep -i aer
[+] Done.
ubuntu@ubuntu:~$ dmesg | grep -i aer
[ 0.605022] acpi PNP0A08:00: _OSC: OS now controls [PME AER PCIeCapability]
[ 0.731034] acpi ACPI0016:00: _OSC: OS now controls [PME AER PCIeCapability]
[ 1.201521] pcieport 0000:0c:00.0: AER: enabled with IRQ 24
[ 1.209685] pcieport 0000:0c:01.0: AER: enabled with IRQ 25
[ 536.761401] pcieport 0000:0c:01.0: aer_inject: Injecting errors 00000000/00000000 into device 0000:0e:00.0
[ 536.761653] pcieport 0000:0c:01.0: AER: Correctable error message received from 0000:0e:00.0
ubuntu@ubuntu:~$
```

7.4 AER_Inject_Uncorrectable_Fatal.sh (Uncorrectable Fatal AER)

AER_Inject_Uncorrectable_Fatal.sh injects an uncorrectable fatal PCIe AER error via /dev/aer_inject using PCI_ERR_UNC_DLP (bit 4, Data Link Protocol Error). A child process maps /dev/dax0.0 and reads CXL memory in a loop while the parent waits 5 seconds then injects the fatal error. The aer-inject kernel module must be loaded first. The script checks and configures the DAX device mode before running. After injection the kernel runs its full AER recovery sequence, resets the PCIe link, and the CXL device goes offline for the remainder of the session.

```
#!/bin/bash

set -e

WORKDIR="$(mktemp -d)"
SRC="$WORKDIR/aer_uncorrectable.c"
BIN="$WORKDIR/aer_uncorrectable"

cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT

if [ "$EUID" -ne 0 ]; then
    echo "[-] Error: Please run this script as root."
    exit 1
fi

if ! command -v gcc >/dev/null 2>&1; then
    echo "[-] Error: gcc not found. Install a compiler (e.g. apt install gcc)."
    exit 1
fi

DAX_DEV="dax0.0"
DAX_PATH="/dev/$DAX_DEV"
REGION="region0"
DECODER="decoder0.0"
MEMDEV="mem1"

get_dax_mode() {
    daxctl list -d "$DAX_DEV" 2>/dev/null | grep -o '"mode":"[a-z-]*"' | cut -d'"' -f4
}

if ! command -v daxctl >/dev/null 2>&1 || ! command -v cxl >/dev/null 2>&1; then
    echo "[-] Error: daxctl/cxl tools not found, cannot set up the DAX device."
    exit 1
fi
```

```

if [ ! -e "$DAX_PATH" ]; then
    echo "[!] $DAX_PATH not found. No CXL region exists yet on this boot."
    echo "    -> Preventing new memory blocks from auto-onlining as system-ram"
    echo offline > /sys/devices/system/memory/auto_online_blocks 2>/dev/null || true

    echo "    -> Creating region on $MEMDEV via $DECODER..."
    if ! cxl create-region -d "$DECODER" -m "$MEMDEV" -t ram; then
        echo "[-] Error: cxl create-region failed. Check decoder/memdev names with:"
        echo "    cxl list -D / cxl list -M"
        exit 1
    fi
    sleep 1
fi

if [ ! -e "$DAX_PATH" ]; then
    echo "[-] Error: $DAX_PATH still not present after creating the region."
    exit 1
fi

DAX_MODE="$(get_dax_mode)"

if [ "$DAX_MODE" != "devdax" ]; then
    echo "[!] $DAX_PATH is in '$DAX_MODE' mode, not 'devdax'. Trying reconfigure..."
    daxctl reconfigure-device --mode=devdax "$DAX_DEV" >/dev/null 2>&1 || true
    DAX_MODE="$(get_dax_mode)"
fi

if [ "$DAX_MODE" != "devdax" ]; then
    echo "[-] Error: $DAX_PATH is stuck in '$DAX_MODE' mode and cannot be safely"
    echo "    switched to devdax without risking a kernel panic."
    echo ""
    echo "    Required fix: power-cycle (hard reset) the VM, then BEFORE running"
    echo "    anything else that touches mem1/region0/dax0.0, run:"
    echo "    echo offline | sudo tee /sys/devices/system/memory/auto_online_blocks"
    echo "    and only then rerun this script."
    exit 1
fi

echo "[+] $DAX_PATH is in devdax mode."

echo "[+] Step 1: Ensuring aer-inject kernel module is loaded"
if ! lsmod | grep -q '^aer_inject'; then
    modprobe aer-inject || {
        echo "[-] Error: could not load aer-inject module."
        exit 1
    }
fi

if [ ! -e /dev/aer_inject ]; then
    echo "[-] Error: /dev/aer_inject not found even after modprobe."
    exit 1
fi

echo "[!] WARNING: this will take the CXL device offline for the rest of the"
echo "    session. A VM restart is expected to be required afterward."
read -r -p "    Continue? [y/N] " CONFIRM
case "$CONFIRM" in
    [yY][yY][eE][sS]) ;;
    *) echo "Aborted."; exit 0 ;;
esac

echo "[+] Step 2: Writing and compiling uncorrectable fatal AER injector"

```

```

cat > "$SRC" << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <signal.h>
#include <stdint.h>
#include <string.h>
#include <sys/wait.h>

#define MAP_SIZE (2 * 1024 * 1024UL)

struct aer_error_inj {
    uint8_t bus;
    uint8_t dev;
    uint8_t fn;
    uint32_t uncor_status;
    uint32_t cor_status;
    uint32_t header_log0;
    uint32_t header_log1;
    uint32_t header_log2;
    uint32_t header_log3;
    uint32_t domain;
};

#define PCI_ERR_UNC_DLP 0x00000010

volatile int sigbus_caught = 0;
volatile int keep_running = 1;

void handle_sigbus(int sig, siginfo_t *info, void *ctx) {
    sigbus_caught++;
    fprintf(stderr, "\n[!] SIGBUS at %p count: %d\n", info->si_addr, sigbus_caught);
}

void run_active_memory() {
    struct sigaction sa = {0};
    sa.sa_sigaction = handle_sigbus; sa.sa_flags = SA_SIGINFO;
    sigaction(SIGBUS, &sa, NULL);
    int fd = open("/dev/dax0.0", O_RDWR);
    if (fd < 0) { perror("open /dev/dax0.0"); exit(1); }
    void *map = mmap(NULL, MAP_SIZE, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); exit(1); }
    printf("[*] Child PID %d: CXL DAX mapped at %p\n", getpid(), map);
    uint64_t *ptr = (uint64_t *)map;
    for (size_t i = 0; i < 512; i++) ptr[i] = 0xDEADBEEF00000000UL | i;
    printf("[*] Child: pattern written, reading in loop...\n");
    fflush(stdout);
    uint64_t passes = 0;
    while (keep_running) {
        for (size_t i = 0; i < 512; i++) {
            uint64_t val = ptr[i];
            if (val != (0xDEADBEEF00000000UL | i))
                printf("[!] Mismatch at %zu: 0x%lx\n", i, val);
        }
        passes++;
        if (passes % 200 == 0) {
            printf("[.] %lu passes SIGBUS: %d\n", passes, sigbus_caught);
            fflush(stdout);
        }
        usleep(1000);
    }
}

```

```

    }
    munmap(map, MAP_SIZE); close(fd);
    exit(sigbus_caught > 0 ? 2 : 0);
}

void inject_uncorrectable_aer() {
    int fd = open("/dev/aer_inject", O_WRONLY);
    if (fd < 0) { perror("open /dev/aer_inject"); return; }
    struct aer_error_inj einj; memset(&einj, 0, sizeof(einj));
    einj.bus = 0x0e; einj.dev = 0x00; einj.fn = 0x00;
    einj.uncor_status = PCI_ERR_UNC_DLP; einj.domain = 0;
    printf("[*] Injecting UNCORRECTABLE FATAL AER on 0000:0e:00.0\n");
    ssize_t ret = write(fd, &einj, sizeof(einj));
    if (ret < 0) perror("write /dev/aer_inject");
    else printf("[*] Uncorrectable AER injection sent.\n");
    close(fd);
}

int main() {
    printf(" CXL Uncorrectable Fatal AER Experiment  \n\n");
    pid_t child = fork();
    if (child == 0) { run_active_memory(); return 0; }
    printf("[*] Parent: waiting 5s while child reads CXL memory...\n");
    sleep(5);
    printf("\n[*] Parent: injecting UNCORRECTABLE FATAL AER\n");
    inject_uncorrectable_aer();
    printf("[*] Parent: waiting 5s post-injection\n");
    sleep(5);
    keep_running = 0; sleep(1);
    kill(child, SIGTERM);
    int status; waitpid(child, &status, 0);
    printf("\n Results  \n");
    if (WIFEXITED(status))
        printf("[*] Child exit: %d %s\n", WEXITSTATUS(status),
            WEXITSTATUS(status) == 2 ? "(SIGBUS caught)" : "(no SIGBUS)");
    else if (WIFSIGNALED(status))
        printf("[!] Child killed by signal: %d\n", WTERMSIG(status));
    printf("[*] Check: dmesg | grep -i aer\n");
    return 0;
}
EOF

gcc -O2 -o "$BIN" "$SRC"

echo "[+] Step 3: Running uncorrectable fatal AER experiment"
"$BIN"

echo "[+] Step 4: Verify device state (expected to be offline until restart):"
echo " cxl list -d mem1"
echo " dmesg | grep -i aer"

echo "[+] Done."

```

Output:

```
[!] SIGBUS at 0x7f8c46200000 count: 6462
[!] SIGBUS at 0x7f8c46200000 count: 6463
[!] SIGBUS at 0x7f8c46200000 count: 6464
[!] SIGBUS at 0x7f8c46200000 count: 6465
[!] SIGBUS at 0x7f8c46200000 count: 6466
[!] SIGBUS at 0x7f8c46200000 count: 6467
[!] SIGBUS at 0x7f8c46200000 count: 6468
[!] SIGBUS at 0x7f8c46200000 count: 6469
[!] SIGBUS at 0x7f8c46200000 count: 6470

=== Results ===
[!] Child killed by signal: 15
[*] Check: dmesg | grep -i aer
[+] Step 4: Verify device state (expected to be offline until restart):
    cxl list -d mem1
    dmesg | grep -i aer
[+] Done.
ubuntu@ubuntu:~$ cxl list -d mem1
Warning: no matching devices found

[]
ubuntu@ubuntu:~$ dmesg | grep -i aer
[  0.584546] acpi PNP0A08:00: _OSC: OS now controls [PME AER PCIeCapability]
[  0.706593] acpi ACPI0016:00: _OSC: OS now controls [PME AER PCIeCapability]
[  1.181301] pcieport 0000:0c:00.0: AER: enabled with IRQ 24
[  1.191050] pcieport 0000:0c:01.0: AER: enabled with IRQ 25
[  86.844726] pcieport 0000:0c:01.0: aer_inject: Injecting errors 00000000/00000010 into device 0000:0e:00.0
[  86.844929] pcieport 0000:0c:01.0: AER: Uncorrectable (Fatal) error message received from 0000:0e:00.0
[  86.844949] cxl_pci 0000:0e:00.0: AER: PCIe Bus Error: severity=Uncorrectable (Fatal), type=Inaccessible, (Unregistered Agent ID)
[  87.822985] pcieport 0000:0c:01.0: AER: Root Port link has been reset (0)
[  87.871181] pcieport 0000:0c:01.0: AER: device recovery successful
ubuntu@ubuntu:~$
```

8. Results

8.1 Active Memory Experiments

Different injection paths produced clearly different outcomes. Correctable AER was handled silently by the kernel with no impact on the running process. Uncorrectable fatal AER triggered the kernel's full recovery sequence, reset the PCIe link, and left the CXL device inaccessible for the rest of the session. Poison injection via both debugfs and raw CCI mailbox was recorded correctly by the firmware and kernel but produced no observable impact on any running process in this QEMU setup.

8.2 Why AER and Poison Behave Differently

On real CXL hardware, poison is consumed at the point of memory access. When a program reads a poisoned address through a direct mapping, the CPU's own memory controller detects the poison the instant the load instruction executes and raises a Machine Check Exception (MCE), a CPU-level hardware exception. The kernel's MCE handler then has to decide what to do about a fault that occurred mid-instruction on a page actively mapped into a running process, which can result in a kernel panic if the page cannot be safely recovered. In the QEMU setup used here, the firmware correctly records the poisoned DPA and the kernel logs the injection event, but the MCE signal on poison consumption was not observed.

AER is consumed at the PCIe link level, independent of whether any specific memory address is being accessed at that moment. The link itself reports a transient problem; Linux's AER driver responds at the link layer, which does not require freezing or revoking any specific page a process has mapped. The memory contents themselves were never marked bad, only the link briefly hiccuped, so an actively-running process is simply unaffected once the link recovers. The difference is about which signaling pathway (CCI/poison+MCE vs PCIe AER) the error travels

down, and whether that pathway's recovery mechanism can operate without disturbing a page currently being read or written by a live process.

8.3 Injection Method Ranking

Method	Who Generates Error	Layer	Authenticity
QMP	QEMU hypervisor from host side	Hypervisor	Lowest, guest is a passive receiver
/dev/aer_inject write()	Guest kernel's PCIe AER module, triggered by guest program	PCIe	Better than QMP, guest-initiated, but PCIe-generic
debugfs inject_poison	Guest CXL driver issues CCI command on behalf of debugfs write	CXL CCI (via kernel)	High, real CCI, kernel-mediate
Raw CCI via /dev/cxl/memX + CXL_MEM_SEND_COMMAND	Userspace program directly sends CCI command to device mailbox	CXL CCI (direct)	Highest, most direct, most authentic

9. General Media, DRAM Memory Module from Guest Side

These three event types are only injectable via QEMU QMP commands in this QEMU version. QEMU hardcodes them as host-side-only, and there is no mailbox path, debugfs path, or any other guest-visible interface that reaches the same code. The kernel is fully ready to receive and trace them, but QEMU never gives the guest a way to ask for them. Confirmed by reading the QEMU source: all three injection functions (qmp_cxl_inject_general_media_event, etc.) are QMP handlers and are called from nowhere else.

10. Summary

QMP injection writes the end result of an error-handling pipeline directly into QEMU's emulated registers, skipping the pipeline entirely, useful for broad coverage but not realistic.

CCI/mailbox injection: whether via the kernel's debugfs helpers or via a raw CXL_MEM_SEND_COMMAND ioctl, runs the real CXL doorbell protocol against the device's mailbox registers, exercising the actual command/response pathway a real driver would use. Poison injected this way is recorded correctly by the firmware and kernel, the inject and clear opcodes both complete with retval 0x0, and the kernel logs the event. The mailbox protocol works correctly end to end.

PCIe AER injection exercises a third, entirely separate pathway, the PCIe link-layer error reporting registers, which Linux's mature AER recovery code handles gracefully without disturbing actively-mapped memory at all.